

7. JSON, Callback, AJAX

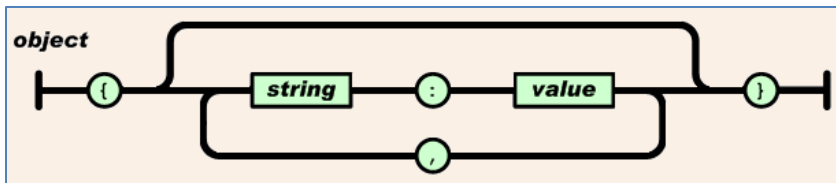
7-1. JSON

7-2. Callback

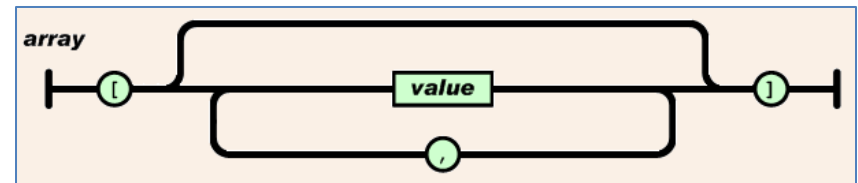
7-3. AJAX

■ JSON (JavaScript Object Notation)

- JavaScript Object Notation
- 간단한 데이터 교환 형식
- 경량 텍스트 기반의 구조
- 읽기 쉬운 key : value 형식
- 특정 언어에 제한적이지 않고 독립적인 방식



```
{  
  "id" : "ggoreb",  
  "age" : 20,  
  "address" : "seoul"  
}
```



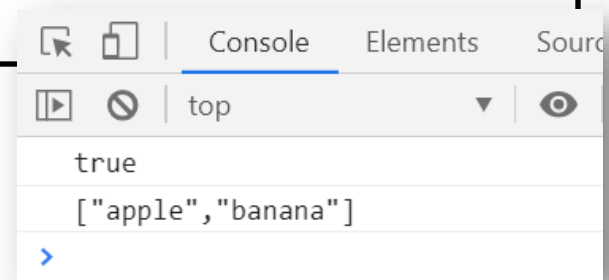
```
[  
  "seoul", "gwanak", "sillim"  
]  
  
{  
  "id" : "ggoreb",  
  "age" : 20,  
  "address" : ["seoul", "gwanak", "sillim"]  
}
```

■ JSON (JavaScript Object Notation)

● Object → JSON (stringify)

07-01-json-stringify.html

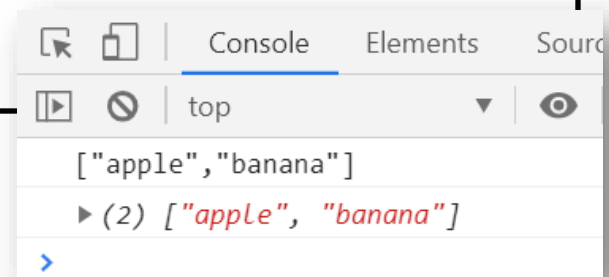
```
let json = JSON.stringify(true);  
console.log(json);  
  
json = JSON.stringify(['apple', 'banana'])  
console.log(json);
```



● JSON → Object (parse)

07-02-json-parse.html

```
let json = JSON.stringify(['apple', 'banana'])  
console.log(json);  
  
const obj = JSON.parse(json);  
console.log(obj);
```

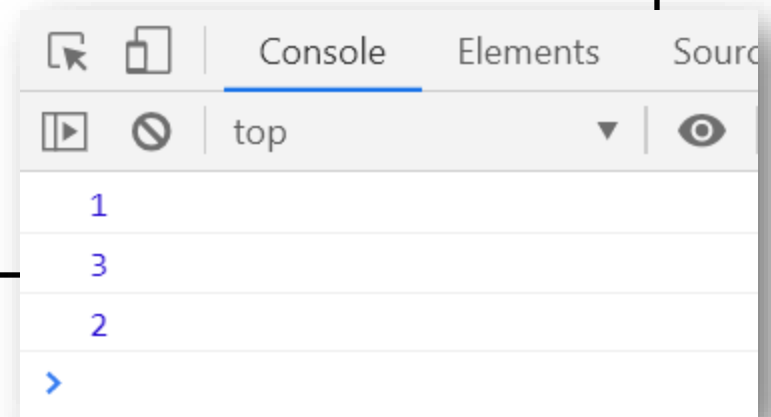


■ Callback

- 특정 시점에 호출되는 함수
- 일반적으로 함수의 매개변수로 전달하도록 설계

07-03-callback.html

```
console.log(1);  
setTimeout(delayPrint, 1000);  
console.log(3);      Callback 함수  
  
function delayPrint() {  
    console.log(2);  
}
```



■ Callback

● 밥 → 설거지

07-04-callback-easy.html

```
<body>
  <div id="result"></div>
  <script>
    const result = document.querySelector("#result");

    function eat(callback) {
      setTimeout(() => {
        console.log('🍴 밥을 먹어요');
        result.innerHTML += `<p>🍴 밥을 먹어요</p>`;
        callback();
      }, 1000);
    }
    function wash() {
      setTimeout(() => {
        console.log('🧼 설거지를 해요');
        result.innerHTML += `<p>🧼 설거지를 해요</p>`;
      }, 1000);
    }
    eat(wash);
  </script>
</body>
```

🍴 밥을 먹어요

🧼 설거지를 해요

■ Callback

● 밥 → 설거지

07-05-callback-hard.html

```
<body>
  <div id="result"></div>
  <script>
    const result = document.querySelector("#result");

    function eat(callback) {
      setTimeout(() => {
        result.innerHTML += `<p>🍴 밥을 먹어요</p>`;
        callback();
      }, 1000);
    }

    function wash() {
      setTimeout(() => {
        result.innerHTML += `<p>🧼 설거지를 해요</p>`;
        callback();
      }, 1000);
    }

    eat(() => wash(
      () => {
        result.innerHTML += `<p>종료</p>`;
      }
    ));
  </script>
</body>
```

🍴 밥을 먹어요

🧼 설거지를 해요

종료

■ Callback

● 밥 → 물 → 그릇 치우기 → 설거지 → 종료

07-06-callback-hell.html

```
<body>
  <div id="result"></div>
  <script>
    const result = document.querySelector("#result");

    function eat(callback) {
      setTimeout(() => {
        result.innerHTML += `<p>🍚 밥을 먹어요</p>`;
        callback();
      }, 1000);
    }

    function drink(callback) {
      setTimeout(() => {
        result.innerHTML += `<p>🥤 물을 마셔요</p>`;
        callback();
      }, 1000);
    }

    function cleanTable(callback) {
      setTimeout(() => {
        result.innerHTML += `<p>🍽️ 밥그릇을 치워요</p>`;
        callback();
      }, 1000);
    }
  </script>
</body>
```

■ Callback

● 밥 → 물 → 그릇 치우기 → 설거지 → 종료

07-06-callback-hell.html

```
function wash(callback) {
  setTimeout(() => {
    result.innerHTML += `<p>🧼 설거지를 해요</p>`;
    callback();
  }, 1000);
}

eat(() => {
  drink(() => {
    cleanTable(() => {
      wash(() => {
        result.innerHTML += `<p>종료</p>`;
      });
    });
  });
});
</script>
</body>
```

🍽️ 밥을 먹어요

🥤 물을 마셔요

🍽️ 밥그릇을 치워요

🧼 설거지를 해요

종료

■ Promise

- 비동기 작업을 간편하게 처리할 수 있도록 도와주는 객체
- 콜백 함수 대신 사용할 수 있는 객체
- 정상 수행 → 성공 메시지 (resolve)
비정상 수행 → 에러 메시지 (reject)
- Promise 사용 시 알아야 할 2가지 사항
 - 상태(State)
 - 수행중 (Pending) → 완료 (Fulfilled)
 - 수행중 (Pending) → 에러 (Rejected)
 - 데이터 제공자와 데이터 사용자 (Producer / Consumer) 구분
- Promise 기본 사용법

```
Promise(executor: (resolve: (value?: any) => void,  
reject: (reason?: any) => void) => void);
```

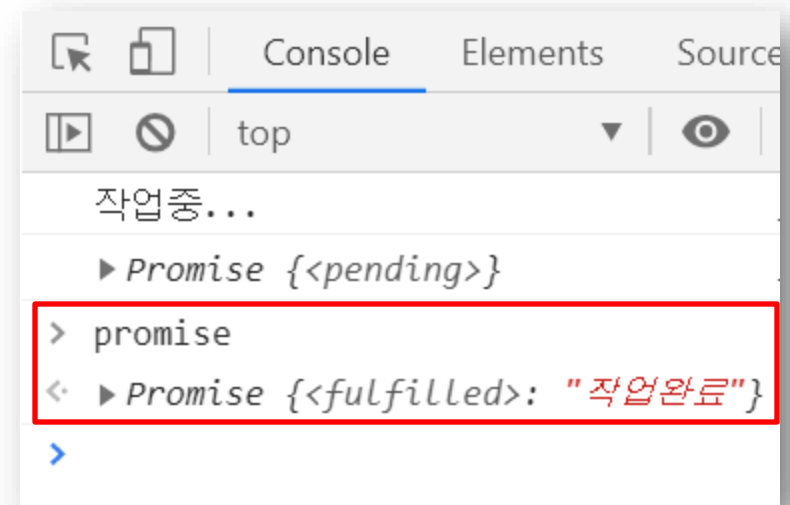
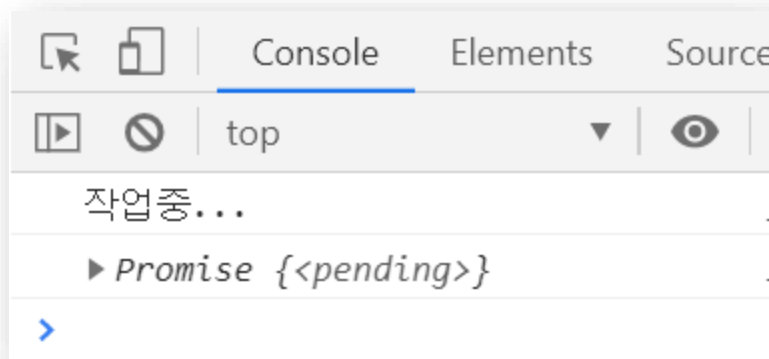
```
const promise = new Promise(function(resolve, reject) {  
  // doing some heavy work (network, read files)  
});
```

■ Promise

● Producer (데이터 제공자) - Promise 객체

07-07-promise.html

```
const promise = new Promise((resolve, reject) => {  
  console.log('작업중...');  
  setTimeout(() => {  
    resolve('작업완료');  
    // reject(new Error('응답없음'));  
  }, 2000);  
});  
console.log(promise);
```



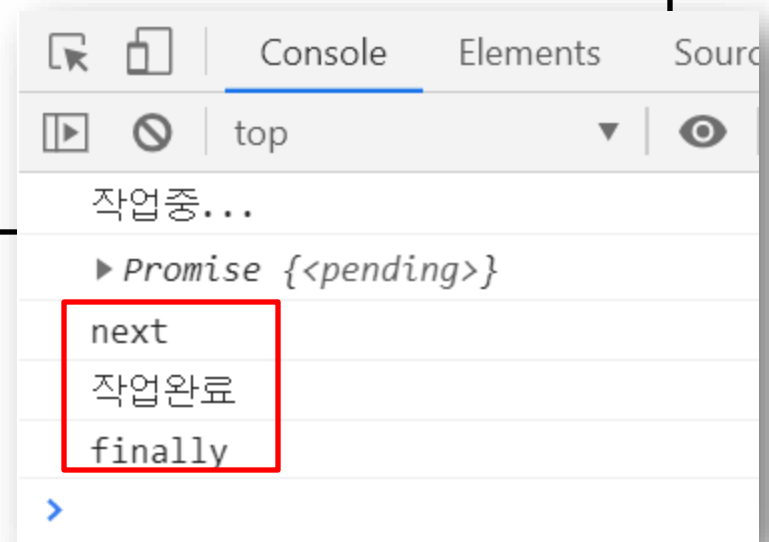
■ Promise

● Consumer (데이터 사용자) - then / catch / finally

07-08-promise.html

```
promise ②
  .then((value) => { // resolve
    console.log(value);
  })
  .catch((error) => {
    console.log(error); // reject
  })
  .finally(() => { ③
    console.log('finally');
  });

console.log('next' ①);
```



■ Promise

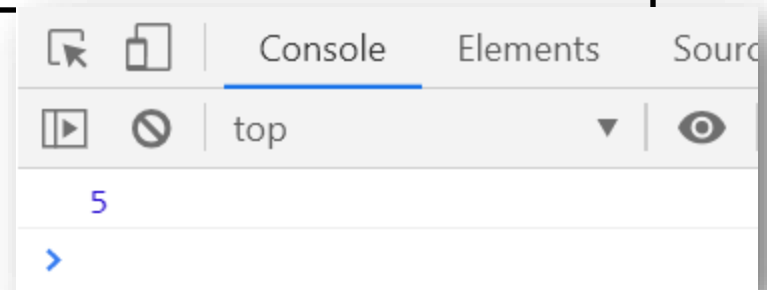
● Chaining

07-09-chaining.html

```
const fetchNumber = new Promise((resolve, reject) => {  
  setTimeout(() => resolve(1), 1000); ①  
})
```

fetchNumber

```
  .then(num => num * 2) ②  
  .then(num => num * 3) ③  
  .then(num => { ④  
    return new Promise((resolve, reject) => {  
      resolve(num - 1); ⑤  
    })  
  })  
  .then(num => console.log(num)); ⑥
```



■ Promise

● 밥 → 설거지

07-10-promise-easy.html

```
<body>
  <div id="result"></div>
  <script>
    const result = document.querySelector("#result");
    function eat() {
      return new Promise((resolve) => {
        setTimeout(() => {
          result.innerHTML += `<p>🍴 밥을 먹어요</p>`;
          resolve();
        }, 1000);
      });
    }
    function wash() {
      return new Promise((resolve) => {
        setTimeout(() => {
          result.innerHTML += `<p>🧼 설거지를 해요</p>`;
          resolve();
        }, 1000);
      });
    }
    eat().then(wash);
  </script>
</body>
```

🍴 밥을 먹어요

🧼 설거지를 해요

■ Promise

● 밥 → 설거지

07-11-promise-hard.html

```
<body>
  <div id="result"></div>
  <script>
    const result = document.querySelector("#result");
    function eat() {
      return new Promise((resolve) => {
        setTimeout(() => {
          result.innerHTML += `<p>🍴 밥을 먹어요</p>`;
          resolve();
        }, 1000);
      });
    }
    function wash() {
      return new Promise((resolve) => {
        setTimeout(() => {
          result.innerHTML += `<p>🧼 설거지를 해요</p>`;
          resolve();
        }, 1000);
      });
    }
    eat()
      .then(wash)
      .then(() => result.innerHTML += `<p>종료</p>`);
  </script>
</body>
```

🍴 밥을 먹어요

🧼 설거지를 해요

■ Promise

● 밥 → 물 → 그릇 치우기 → 설거지 → 종료

07-12-promise-hell.html

```
<body>
  <div id="result"></div>
  <script>
    const result = document.querySelector("#result");

    function eat() {
      return new Promise((resolve) => {
        setTimeout(() => {
          result.innerHTML += `<p>🍴 밥을 먹어요</p>`;
          resolve();
        }, 1000);
      });
    }

    function drink() {
      return new Promise(resolve => {
        setTimeout(() => {
          result.innerHTML += `<p>🥤 물을 마셔요</p>`;
          resolve();
        }, 1000);
      });
    }
  </script>
</body>
```

■ Promise

● 밥 → 물 → 그릇 치우기 → 설거지 → 종료

07-12-promise-hell.html

```
function cleanTable() {
  return new Promise(resolve => {
    setTimeout(() => {
      result.innerHTML += `<p>🍽 밥그릇을 치워요</p>`;
      resolve();
    }, 1000);
  });
}

function wash() {
  return new Promise((resolve) => {
    setTimeout(() => {
      result.innerHTML += `<p>🧼 설거지를 해요</p>`;
      resolve();
    }, 1000);
  });
}

eat()
  .then(drink)
  .then(cleanTable)
  .then(wash)
  .then(() => result.innerHTML += `<p>종료</p>`);
</script>
</body>
```

■ 연습문제 (07-연습문제1.html)

- 아래의 코드를 Promise와 then() 체이닝 방식으로 수정하기
- 출력 예시 (1초 간격)

- 
-  >> 
-  >>  >> 

```
<body>
  <div id="show"></div>
  <script>
    const show = document.querySelector("#show");
    const getHen = () => {
      setTimeout(() => show.innerHTML += ' , 1000);
    };
    const getEgg = () => {
      setTimeout(() => show.innerHTML += ' >> , 1000);
    };
    const cook = (egg) => {
      setTimeout(() => show.innerHTML += ' >> , 1000);
    };
    getHen();    getEgg();    cook();
  </script>
</body>
```

■ AJAX

- **A**synchronous **J**avaScript **A**nd **X**ML

- 비동기 통신

- 활용 예)

- 구글 / 네이버 지도

- 유튜브 영상 재생 중 댓글 달기

- 가입을 위한 정보 입력 중 아이디 중복 검사

- 포털 사이트에서 검색어 입력 중 추천 검색어 제시 등

■ AJAX

● JavaScript Old API

```
const xhr = new XMLHttpRequest();  
xhr.onreadystatechange = 콜백 함수;  
xhr.open('메소드', '주소');  
xhr.send(formData);
```

● JavaScript New API (fetch)

```
const ajax = fetch('주소', { 옵션 });  
ajax.then(콜백 함수);
```

● jQuery

```
<script src='CDN 주소'></script>  
  
$.ajax({  
  url: '주소',  
  type: '메소드',  
  data: Query Parameter  
  success: 콜백 함수  
});
```

● Axios

```
<script src='CDN 주소'></script>  
  
const data = axios({  
  url: '주소',  
  method: 'get',  
  params: Query Parameter  
});  
data.then(콜백 함수);
```

■ AJAX

● JavaScript New API (GET)

```
<script>
  const ajax = fetch('주소');
  ajax.then(function(response) {
    return response.text();
  }).then(function(result) {
    console.log(result);
  });
</script>
```

■ AJAX

● JavaScript New API (GET)

07-13-ajax-get.html

```
<div id="content"></div>

<script>
  const ajax = fetch('http://ggoreb.com/ajax/html/data1.jsp');
  ajax.then(res => {
    return res.text();
  }).then(result => {
    console.log(result);
    document.querySelector('#content').innerHTML += result;
  });
</script>
```

아이디
비밀번호

```
▼ <div id="content">
  <h1>아이디</h1>
  <h1>비밀번호</h1>
</div>
```

■ AJAX

● JavaScript New API (POST - Form Data)

```
<script>
  const formData = new FormData();
  formData.append('key', 'value');
  const ajax = fetch('주소', {
    method: 'post', body: formData
  });
  ajax.then(function(response) {
    return response.text();
  }).then(function(result) {
    console.log(result);
  });
</script>
```

■ AJAX

● JavaScript New API (POST)

07-14-ajax-post.html

```
<div id="content"></div>

<script>
  const formData = new FormData();
  const ajax = fetch(
    'http://ggoreb.com/ajax/html/data2.jsp',
    { method: 'post', body: formData }
  );
  ajax.then(res => {
    return res.text();
  }).then(result => {
    console.log(result);
    document.querySelector('#content').innerHTML += result;
  });
</script>
```

- A
- B

```
<div id="content">
  <ul>
    <li>...</li>
    <li>...</li>
  </ul>
</div>
```

■ AJAX

● JavaScript New API (POST - JSON Data)

```
<script>
  const formData = new FormData();
  formData.append('key', 'value');

  const ajax = fetch('주소', {
    method: 'post',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(Object.fromEntries(formData))
  });
  ajax.then(function(response) {
    return response.json();
  }).then(function(result) {
    console.log(result);
  });
</script>
```

■ AJAX

● 공통 데이터

07-15-ajax-dinosaur.html

```
<head>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css
/bootstrap.min.css" rel="stylesheet">
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js
/bootstrap.bundle.min.js"></script>

  <style>
    .dropdown { display: inline-block; }
    .dropdown + .dropdown { margin-left: 10px; }
  </style>

</head>

<body>
  <div class="container mt-3" id="content"></div>
```

■ AJAX

● 공통 데이터

07-15-ajax-dinosaur.html

```
<script>
  const url = 'http://ggoreb.com/dinosaur/info.jsp';
  const ajax = fetch(url, {
    method: 'get'
  })
    .then(res => res.json())
    .then(data => {
      let code = '';
      for(const item of data) {
        code += `<div class="dropdown">
                  <button class="btn btn-primary dropdown-toggle"
                        data-bs-toggle="dropdown">
                      ${item.kind} (${item['dinoosaurs'].length})
                  </button>`;
        code += `<ul class="dropdown-menu">`;
        for(const dinosaur of item['dinoosaurs']) {
          code += `<li><a class="dropdown-item" href="#">
                    ${dinosaur.title}</a></li>`;
        }
        code += `</ul></div>`;
      }
      document.querySelector('#content').innerHTML = code;
    });
</script>
</body>
```

- 연습문제 (07-연습문제2.html)
 - 야간시간 운영 약국 정보 출력하기
 - 출력 예시 (약국명, 전화번호, 주소)

세종시 야간 약국 정보

약국명	전화번호	주소
제일약국	031-885-2482	경기도 여주시 세종로 27 (홍문동)
정원약국	02-755-5866	서울특별시 중구 세종대로18길 10, (태평로2가)
우리들약국	031-885-5121	경기도 여주시 세종로 73
아람약국	044-862-5265	세종특별자치시 보듬3로 143, 대영프라자 102호 (아름동)
세종약국	043-213-0068	충청북도 청주시 청원구 오창읍 과학산업3로 202-7, 1층 102~103호
세종약국	041-579-3675	충청남도 천안시 동남구 서부대로 276 , 101호 (신방동)
세종약국	051-758-4915	부산광역시 수영구 광안해변로277번길 3, (민락동)
보룡약국	031-886-5147	경기도 여주시 세종로 77, 1층 (홍문동)
대우사약국	02-753-2703	서울특별시 중구 세종대로 22 (남대문로5가)

■ 연습문제 (07-연습문제2.html)

● API URL

https://ggoreb.pythonanywhere.com/night_pharmacy/data/?search=세종

● 기본 코드

```
<head>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css
/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <div class="container mt-4">
    <h2>세종시 야간 약국 정보</h2>
    <div id="content"></div>
  </div>

  <script>
    const url =
      'https://ggoreb.pythonanywhere.com/night_pharmacy/data/?search=세종';

    // 코드 작성

  </script>
</body>
```

■ 연습문제 (07-연습문제2.html)

● Table HTML 코드

```
<table class="table table-bordered table-striped">
  <thead class="table-dark">
    <tr>
      <th>약국명</th>
      <th>전화번호</th>
      <th>주소</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>00약국</td>
      <td>051-1234-5678</td>
      <td>부산광역시 해운대구 000</td>
    </tr>
  </tbody>
</table>
```

■ async / await

- 동기 방식처럼 코드를 작성할 수 있도록 만든 새로운 문법
 - async + await
- promise + then + then + ... + then 을 간단하게 표현

```
function fetchUser() {  
  console.log('작업중...');  
  return new Promise((resolve, reject) => {  
    resolve('작업완료');  
  });  
}  
const user = fetchUser();  
user.then(result => console.log(result))
```



07-16-async-await.html

```
async function fetchUser() {  
  console.log('작업중...');  
  return '작업완료';  
}  
const user = fetchUser();  
user.then(result => console.log(result))
```

■ async / await

● async

- 함수 앞에 붙이면 항상 Promise를 반환함
- 내부에서 await 사용 가능

● await

- Promise가 해결될 때까지 기다림
- await 뒤에는 Promise가 오는 것이 기본
- await는 async 함수 안에서만 사용 가능

■ async / await

● async - 반환값 없음

07-17-async-await.html

```
<script>
  async function start() {
    // 아무 것도 없음
  }

  const result = start();
  console.dir(result); // Promise {<fulfilled>: undefined}
</script>
```

- async가 붙은 함수는 무조건 **Promise**를 반환
- return이 없더라도 Promise.resolve(undefined)처럼 동작함.

■ async / await

● async - 반환값 사용

07-18-async-await.html

```
<script>
  async function start() {
    return '👋 hello';
  }

  const result = start();
  console.dir(result); // Promise {<fulfilled>: '👋 hello'}

  result.then(msg => console.log('✅ 결과:', msg));
  // ✅ 결과: 👋 hello

</script>
```

- async 함수 내부에서 값을 반환하면 자동으로 **Promise**를 반환
- then을 사용해서 값 확인

■ async / await

● await

07-19-async-await.html

```
<script>
  async function sayHi() {
    return '안녕하세요!';
  }

  async function show() {
    const msg = await sayHi();
    console.log(msg); // 출력: 안녕하세요!
  }

  show();
</script>
```

- Promise가 끝날 때까지 기다렸다가 결과 확인
- await는 async 함수 안에서만 사용 가능

■ async / await

● await

07-20-async-await-settimeout.html

```
<script>
  function waitOneSecond() {
    return new Promise(resolve => {
      setTimeout(() => {
        console.log('✅ 1초 경과');
        resolve('🎉 완료!');
      }, 1000);
    });
  }

  async function run() {
    console.log('⌚ 대기 중...');
    const result = await waitOneSecond();
    console.log(result);
  }

  run();
</script>
```

- setTimeout과 같은 콜백 함수를 사용하려면 Promise로 감싸야 가능

■ Promise + then → async + await

07-21-async-await.html

Promise + then

```
const promise = new Promise((resolve, reject) => {  
  console.log('작업중...');  
  setTimeout(() => {  
    resolve('작업완료');  
    // reject(new Error('응답없음'));  
  }, 2000);  
});
```

```
promise  
  .then((value) => { // resolve  
    console.log(value);  
  })  
  .catch((error) => {  
    console.log(error); // reject  
  })  
  .finally(() => {  
    console.log('finally');  
  });
```

```
console.log('next');
```

async + await

```
const promise = new Promise((resolve, reject) => {  
  console.log('작업중...');  
  setTimeout(() => {  
    resolve('작업완료');  
    // reject(new Error('응답없음'));  
  }, 2000);  
});
```

```
const run = async () => {  
  try {  
    const value = await promise;  
    console.log(value);  
  } catch(error) {  
    console.log(error);  
  } finally {  
    console.log('finally');  
  }  
};
```

```
run();
```

```
console.log('next');
```

■ Promise + then → async + await

07-22-async-await.html

Promise + then

```
const fetchNumber =  
  new Promise((resolve, reject) => {  
    setTimeout(() => resolve(1), 1000);  
  })  
  
fetchNumber  
  .then(num => num * 2)  
  .then(num => num * 3)  
  .then(num => {  
    return new Promise((resolve, reject) => {  
      setTimeout(() => resolve(num - 1));  
    })  
  })  
  .then(num => console.log(num));
```

async + await

```
const fetchNumber =  
  new Promise((resolve, reject) => {  
    setTimeout(() => resolve(1), 1000);  
  })  
  
const run = async () => {  
  let num = await fetchNumber;  
  num = num * 2;  
  num = num * 3;  
  return num - 1;  
}  
  
run().then(num => console.log(num));
```

■ Promise + then → async + await

● 밥 → 물 → 그릇 치우기 → 설거지 → 종료

07-23-async-await.html

```
<body>
  <div id="result"></div>
  <script>
    const result = document.querySelector("#result");

    function eat() {
      return new Promise((resolve) => {
        setTimeout(() => {
          result.innerHTML += `<p>🍴 밥을 먹어요</p>`;
          resolve();
        }, 1000);
      });
    }

    function drink() {
      return new Promise(resolve => {
        setTimeout(() => {
          result.innerHTML += `<p>🥤 물을 마셔요</p>`;
          resolve();
        }, 1000);
      });
    }
  </script>
</body>
```

■ Promise + then → async + await

● 밥 → 물 → 그릇 치우기 → 설거지 → 종료

07-23-async-await.html

```
function cleanTable() {  
  return new Promise(resolve => {  
    setTimeout(() => {  
      result.innerHTML += `

 밥그릇을 치워요</p>`;   
      resolve();  
    }, 1000);  
  });  
}  
  
function wash() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      result.innerHTML += `

 설거지를 해요</p>`;   
      resolve();  
    }, 1000);  
  });  
}  
  
</script>  
</body>


```

■ Promise + then → async + await

● 밥 → 물 → 그릇 치우기 → 설거지 → 종료

07-23-async-await.html

Promise + then

```
eat()  
  .then(drink)  
  .then(cleanTable)  
  .then(wash)  
  .then(() => {  
    result.innerHTML += `

종료

`  
  });
```

async + await

```
async function show() {  
  await eat();  
  await drink();  
  await cleanTable();  
  await wash();  
  result.innerHTML += `

종료

`  
};  
  
show();
```

■ 연습문제 (07-연습문제3.html)

- "07-15-ajax-dinosaur.html"의 코드에 async와 await를 적용해서 동일한 결과가 출력될 수 있도록 수정하기

```
<script>

const url = 'http://ggoreb.com/dinosaur/info.jsp';

const ajax = fetch(url, {
  method: 'get'
});

</script>
```